

MLOps Project for a Mask R-CNN on GCP using uWSGI Flask

Business Overview

In the field of deploying and automating machine learning solutions, MLOps plays a vital role in making models scalable, reliable, and continuously adaptable to production needs. The key to using Google Cloud services such as Cloud Build, Artifact Registry, Cloud Run, and GKE allows organizations to streamline their ML workflows and focus on delivering impactful insights and business value.

This project demonstrates an end-to-end MLOps pipeline to deploy a Mask R-CNN model for Image Segmentation as a web application using Flask with uWSGI, Docker, and TensorFlow. We recommend having a basic understanding of the previous project [Image Segmentation using Mask R-CNN with Tensorflow](#), as this forms the foundation for the deployment process.

Aim:

The project aims to automate the deployment of the machine learning model using Google Cloud Platform (GCP) services, which enables efficient model packaging, deployment, and monitoring through CI/CD workflows. The benefit of building MLOps pipelines using GCP services, such as Cloud Build, Artifact Registry, Cloud Run Function, GCP Pub/Sub, and GKE, the pipeline ensures reproducibility and seamless continuous integration for managing ML model deployment.

Prerequisite Project:

- [Image Segmentation using Mask R-CNN with Tensorflow](#)

Prerequisites:

It is recommended to have a basic knowledge of the following:

1. Python
2. Flask
3. Basic understanding of deployment
4. uWSGI
5. Docker
6. Basics of GCP

Data Description:

For this project, sample image files will be utilized as test data to validate the deployed application running on Kubernetes. The objective is to verify whether the system can successfully process the input images and return the segmentation results in real time. The application supports common image formats such as JPG, PNG, and TIFF.

Tech Stack:

→ **Language:** Python

→ **Services:** Google Cloud Build, Artifact Registry, Pub/Sub, Google Kubernetes Engine (GKE), Cloud Run, Docker

→ **Libraries:** TensorFlow, PyTorch, Matterport Mask R-CNN (mrcnn), Matplotlib, Flask, uWSGI, OS

Note: We recommend monitoring GCP Services and deleting those that are not in use after project execution.

Approach:

In this project, the MLOps pipeline will automate the deployment and management of containerized ML applications using Google Cloud Platform (GCP) services.

1. Local Development and Dockerization

- The ML application was initially developed and tested locally, where all dependencies were installed and verified.
- A Docker container was created to package the model, Flask API, and all required libraries into a portable runtime environment.

2. Source Control and Pipeline Triggering

- In the first stage, all ML application code, configuration files, and deployment scripts were stored in a GitHub repository.
- Every new commit to the repository automatically triggers the CI/CD pipeline configured in Google Cloud Build.
- Since a large model file cannot be stored in GitHub, it was uploaded to Google Cloud Storage (GCS), which Cloud Build accessed during the build process to include the model in the Docker image.

3. Build Stage (Cloud Build, Artifact Registry, Pub/Sub)

- In the second stage, Google Cloud Build was used to build the Docker image for the ML application and push it to Artifact Registry.
- Artifact Registry served as the centralized container registry for storing and managing Docker images.
- After a successful build, Cloud Build published a message to a Pub/Sub topic, containing metadata about the new Docker image.

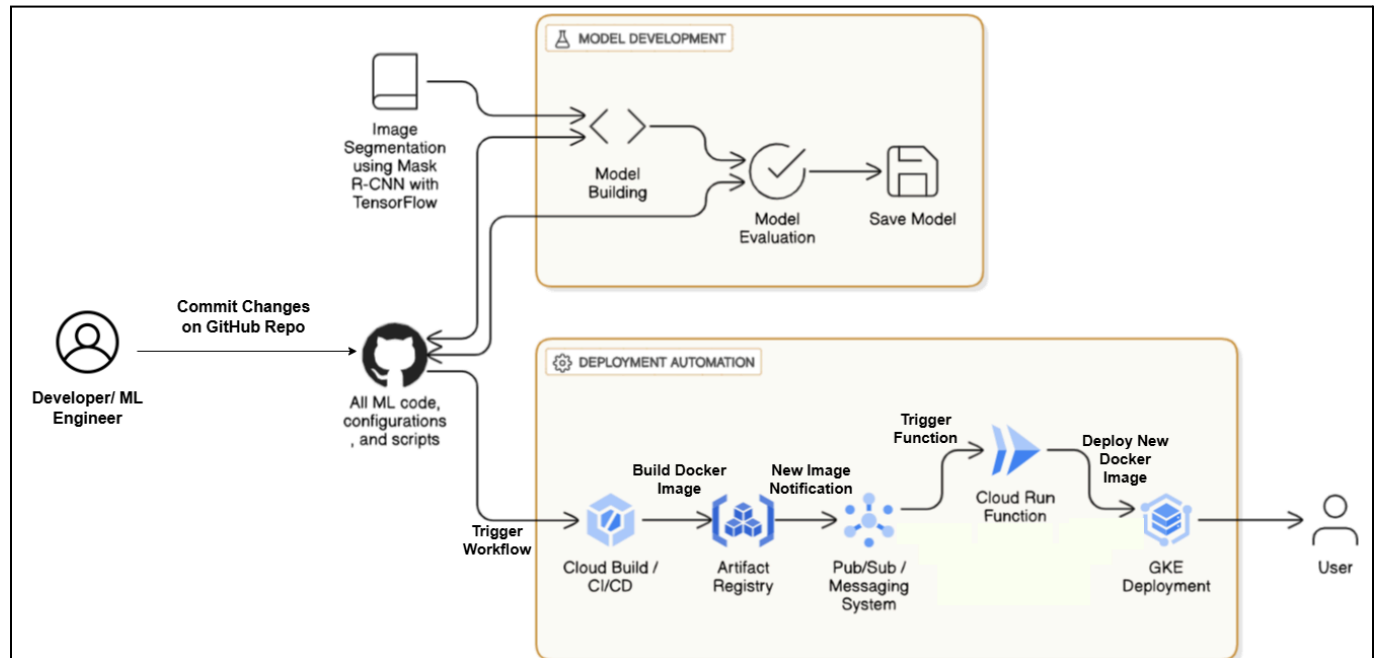
4. Deploy Stage (Cloud Run Function & GKE)

- In the third stage, a Cloud Run Function was configured to listen to Pub/Sub messages and automatically pull the latest Docker image from Artifact Registry.
- The function updated the running application on Google Kubernetes Engine (GKE) by deploying the new image version to active pods.
- This automated update process ensured that every code or model change was seamlessly deployed without manual intervention.

5. Continuous Integration and deployment

- Every commit initiates a full CI/CD cycle, automation, building, and deployment on GCP.

Architecture Diagram:



Key Takeaways:

- Understanding of the MLOps workflow for automating ML model deployment
- How to containerize and package ML applications using Docker?
- Implementation of automated building and publishing of an image to the Artifact Registry using Cloud Build
- Creation of a GCS bucket for storing large ML models to be included in the build
- How to create a GKE cluster using Google Cloud Shell
- How to create a Pub/Sub trigger in Cloud Run Functions for new deployments?
- Learn how Pub/Sub asynchronous and event-driven communication works
- Understanding of Cloud Run Function logic to automate GKE deployment using Pub/Sub triggers?
- Implementation of CI/CD automation using Cloud Build triggers on code changes committed to GitHub
- How to validate ML deployments on GKE through POST and GET requests